

# Financial Vault

## High-Level Design Document

Project 11 · Information Security · Faculty of Engineering

### 1. Introduction

Financial Vault is a web-based application for protecting sensitive financial documents using industry-standard symmetric encryption. It allows users to encrypt files with one of three cryptographic algorithms, store them in a local vault, decrypt and download them when needed, verify their integrity without downloading, and review a complete audit trail of every operation. The system is designed with layered security: passwords are never used as keys directly, every encryption is randomised with a fresh salt and IV, and a SHA-256 fingerprint is stored alongside the ciphertext to detect tampering.

### 2. System Overview

The application is a single-server Python/Flask web application. The user interacts with it through a standard browser. There is no database for user accounts; the vault is a local directory of binary files, and the audit log is a SQLite database. The system is intentionally self-contained and requires no external services.

### 3. Components

| Component       | File                                 | Responsibility                                                                                     |
|-----------------|--------------------------------------|----------------------------------------------------------------------------------------------------|
| Browser / UI    | templates/*.html<br>static/style.css | Renders forms, displays flash messages and audit log. Sends HTTP POST requests.                    |
| Flask Router    | app.py                               | Maps HTTP routes to handler functions. Reads form data, calls crypto/audit/lockout modules.        |
| Crypto Engine   | crypto.py                            | Implements AES-256-CBC, 3DES, ChaCha20 encryption and decryption. PBKDF2 password hashing.         |
| Audit Logger    | audit.py                             | Writes and reads the SQLite audit_log table. Records action, filename, algorithm, success/failure. |
| Lockout Manager | lockout.py                           | Tracks per-file failed decryption attempts in lockout.json. Locks files after 5 failures.          |
| Vault Storage   | vault/*.enc                          | Binary files containing the encrypted bundle (see Section 5).                                      |

## 4. Architecture Diagram

The diagram below shows the layered architecture and the direction of data flow between components.



## 5. Encrypted Bundle Format (.enc)

Each vaulted file is a self-describing binary bundle. All metadata needed to decrypt is stored inside the file — no separate index or manifest is required.

| Offset | Length    | Content                                                    |
|--------|-----------|------------------------------------------------------------|
| 0      | 2 bytes   | Unsigned big-endian short: length of algorithm string (N)  |
| 2      | N bytes   | Algorithm name as UTF-8 text (e.g. 'AES-256-CBC')          |
| 2+N    | 16 bytes  | Random PBKDF2 salt                                         |
| 18+N   | 2 bytes   | Unsigned big-endian short: length of IV/nonce (M)          |
| 20+N   | M bytes   | Random IV (16 B for AES/3DES) or nonce (12 B for ChaCha20) |
| 20+N+M | 64 bytes  | SHA-256 hex digest of the original plaintext               |
| 84+N+M | remainder | Ciphertext                                                 |

## 6. Key Data Flows

---

## 6.1 Encryption Flow

1. Browser sends: file bytes + password + algorithm choice (HTTP POST /encrypt).
2. Flask reads the file bytes and calls `crypto.encrypt_file(data, password, algorithm)`.
3. `crypto.py` generates a 16-byte random salt using `os.urandom`.
4. PBKDF2-HMAC-SHA256 derives the encryption key from the password and salt (100,000 iterations).
5. A random IV (or nonce for ChaCha20) is generated; the file is encrypted.
6. SHA-256 is computed on the original plaintext (before encryption).
7. The bundle is assembled and written to `vault/.enc`.
8. `audit.log()` records the operation; Flask returns a result page showing the SHA-256 fingerprint.

## 6.2 Decryption Flow

1. Browser sends: selected .enc filename + password (HTTP POST /decrypt).
2. Flask checks `lockout.is_locked(filename)`. If locked, returns error page immediately.
3. Flask calls `load_from_vault()` to unpack: algorithm, salt, IV, stored SHA-256, ciphertext.
4. `crypto.decrypt_file()` re-derives the key with PBKDF2 (same password + stored salt).
5. Ciphertext is decrypted to plaintext. On wrong password, a padding/value error is raised.
6. On failure: `lockout.record_failure()` increments the counter; after 5 failures the file is locked.
7. On success: SHA-256 is recomputed on the plaintext and compared to the stored hash.
8. If hashes match: `lockout.record_success()` resets counter; file is sent as a download.
9. `audit.log()` records the outcome (success / failed / tampered).

## 6.3 Verify Flow

Identical to decryption steps 1-7, but instead of streaming the file to the browser, Flask renders a result page showing the stored SHA-256 fingerprint and whether it matched. No plaintext leaves the server.

---

## 7. Messages & Data Exchanged

| Route         | Direction        | Data Sent                                     | Data Returned                        |
|---------------|------------------|-----------------------------------------------|--------------------------------------|
| POST /encrypt | Browser → Server | file (bytes), password (str), algorithm (str) | Result.html with SHA-256 fingerprint |
| POST /decrypt | Browser → Server | enc_file (str), password (str)                | File download (binary) or error page |

|                   |                  |                                |                                      |
|-------------------|------------------|--------------------------------|--------------------------------------|
| POST /verify      | Browser → Server | enc_file (str), password (str) | result.html with hash match status   |
| POST /delete      | Browser → Server | enc_file (str)                 | Redirect to index with flash message |
| POST /unlock      | Browser → Server | enc_file (str)                 | Redirect to audit log                |
| GET /audit        | Browser → Server | (none)                         | audit.html with all log rows         |
| POST /audit/clear | Browser → Server | (none)                         | Redirect to audit log                |

## 8. Security Design Decisions

---

### PBKDF2 Key Derivation

Passwords are never used directly as encryption keys. PBKDF2-HMAC-SHA256 with 100,000 iterations is used to derive a key from the password and a random salt. This makes dictionary and brute-force attacks computationally expensive even if the vault files are stolen.

### Per-Encryption Randomness

A new 16-byte salt is generated for every encryption operation. The IV (or nonce) is also freshly randomised. This means encrypting the same file twice with the same password produces completely different ciphertext, preventing pattern analysis.

### Pre-Encryption Hashing

SHA-256 is computed on the plaintext before encryption and stored inside the bundle. On decryption the hash is recomputed and compared. Any modification to the ciphertext — even a single bit flip — will produce a different plaintext whose hash will not match.

### Lockout Mechanism

Failed decryption attempts are counted per file in lockout.json. After 5 consecutive failures the file is locked, preventing further attempts. A manual unlock from the audit log is required.

### No Secret Storage

Passwords are never stored anywhere — not in the vault files, not in the database, not in memory beyond the lifetime of a single request. There is no user account system.